

# Natural Language Processing (NLP)

## Question Bank

### UNIT -1

Q1. State pre-processing. Compose python code for most frequent words in a document after pre-processing.

```
from nltk.probability import FreqDist
from nltk.tokenize import word_tokenize

text = "data science is fun and data science is useful"
tokens = word_tokenize(text)

fdist = FreqDist(tokens)
print(fdist.most_common(5))
```

Q2. Build a python code simple text classifier using Naive Bayes.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

texts = ["I love data science", "I hate bugs", "Data is powerful", "Debugging is hard"]
labels = [1, 0, 1, 0]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts)

model = MultinomialNB()
model.fit(X, labels)

test = vectorizer.transform(["I love debugging"])
print(model.predict(test))
```

Q3. Develop python code for syntactic parsing using a grammar.

```
import nltk

grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> DT NN
VP -> VB NP
DT -> 'the'
NN -> 'cat' | 'dog'
VB -> 'chased'
""")
```

```

parser = nltk.ChartParser(grammar)
sentence = "the cat chased the dog".split()

for tree in parser.parse(sentence):
    print(tree)

```

Q4. Summarize Basics of Text Mining in NLP with real-world applications.

## UNIT – 2

Q1. Explain importance of TF-IDF in NLP programming. Write python code for converting text into TF-IDF features.

```

from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["data science", "machine learning", "data mining"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)

print(vectorizer.get_feature_names_out())
print(X.toarray())

```

Q2. State ZIP's Law working principle. Write python code to verify Zipf's Law by plotting rank vs frequency.

```

import matplotlib.pyplot as plt
from collections import Counter

text = "data science data mining machine learning data science"
words = text.split()

freq = Counter(words)
values = sorted(freq.values(), reverse=True)

plt.plot(values)
plt.xlabel("Rank")
plt.ylabel("Frequency")
plt.title("Zipf's Law")
plt.show()

```

Q3. Extract implicit product features from customer reviews using dependency-based keywords or frequent noun phrases.

```

import spacy
nlp = spacy.load("en_core_web_sm")

text = "The phone is too slow but the battery lasts long."

doc = nlp(text)

features = [token.text for token in doc if token.pos_ == "NOUN"]
print("Implicit Features:", features)

```

Q4. Use string kernel technique to find similarity between relation patterns with python code.

```

from sklearn.metrics.pairwise import linear_kernel
from sklearn.feature_extraction.text import CountVectorizer

texts = ["John works at Google", "Alice works at Microsoft"]

vectorizer = CountVectorizer(analyzer='char')
X = vectorizer.fit_transform(texts)

similarity = linear_kernel(X, X)
print(similarity)

```

### UNIT – 3

Q1. Classify text using Logistic Regression with python code.

```

from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["good movie", "bad movie", "excellent film", "poor film"]
labels = [1, 0, 1, 0]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)

model = LogisticRegression()
model.fit(X, labels)

print(model.predict(vectorizer.transform(["good film"])))

```

Q2. Consider the input as docs = ["data science", "machine learning", "football match", "soccer game"]. Evaluate clustering quality using silhouette score.

```

from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans

```

```

from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["data science", "machine learning", "football match", "soccer game"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)

kmeans = KMeans(n_clusters=2)
labels = kmeans.fit_predict(X)

print("Silhouette Score:", silhouette_score(X, labels))

```

Q3. Implement a rule-based classifier using keywords.

```

def classify(text):
    if "good" in text or "excellent" in text:
        return "Positive"
    elif "bad" in text or "poor" in text:
        return "Negative"
    else:
        return "Neutral"

print(classify("This is a good product"))

```

Q4. State Bag-of-words. Represent documents using TF-IDF and compare with Bag-of-Words.

```

from sklearn.feature_extraction.text import TfidfVectorizer

docs = ["data science", "data mining", "machine learning"]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)

print("TF-IDF Matrix:\n", X.toarray())

```

## UNIT – 4

Q1. Tokenize a given text into unigrams, bigrams, and trigrams with python code.

```

from nltk.util import ngrams
from nltk.tokenize import word_tokenize

text = "data science is very useful"
tokens = word_tokenize(text)

```

```
print("Unigrams:", list(ngrams(tokens, 1)))
print("Bigrams:", list(ngrams(tokens, 2)))
print("Trigrams:", list(ngrams(tokens, 3)))
```

Q2. State bigram network visualization. Visualize bigram relationships as a network graph with python code.

```
import networkx as nx
import matplotlib.pyplot as plt
from nltk.util import ngrams

text = "data science is fun and data science is powerful"
tokens = text.split()

bigrams = list(ngrams(tokens, 2))

G = nx.Graph()
G.add_edges_from(bigrams)

nx.draw(G, with_labels=True)
plt.show()
```

Q3. Divide text into sections and analyze word frequency correlations.

```
import pandas as pd

sections = [
    "data science is fun",
    "machine learning is powerful"
]

word_counts = []

for sec in sections:
    counts = {}
    for word in sec.split():
        counts[word] = counts.get(word, 0) + 1
    word_counts.append(counts)

df = pd.DataFrame(word_counts).fillna(0)
print(df.corr())
```

Q4. Explain the concept of N-gram tokenization. Describe how unigrams, bigrams, and trigrams are generated from text. Discuss their importance in text mining with suitable examples.

## UNIT – 5

Q1. Apply Latent Dirichlet Allocation (LDA) to extract topics from a corpus with python code.

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

docs = ["data science is fun", "machine learning is powerful", "data mining
techniques"]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

lda = LatentDirichletAllocation(n_components=2, random_state=0)
lda.fit(X)

print("Topics:\n", lda.components_)
```

Q2. Implement Hidden Markov Model (HMM) for sequence labelling with python code.

```
from hmmlearn import hmm
import numpy as np

# Example sequence
X = np.array([[0], [1], [2], [1], [0]])

model = hmm.MultinomialHMM(n_components=2)
model.fit(X)

hidden_states = model.predict(X)
print("Hidden states:", hidden_states)
```

Q3. Parse a sentence using probabilistic Context-Free Grammar (CFG) with python code.

```
import nltk

grammar = nltk.PCFG.fromstring("""
S -> NP VP [1.0]
NP -> 'dog' [0.5] | 'cat' [0.5]
VP -> 'runs' [0.5] | 'jumps' [0.5]
""")

parser = nltk.ViterbiParser(grammar)
```

```
sentence = ["dog", "runs"]
```

```
for tree in parser.parse(sentence):  
    print(tree)
```

Q4. Discuss the concept of **word-topic probabilities** in LDA. How are these probabilities used for **per-document topic classification**? Explain how words are assigned to topics.